

Práctico 1 - Punto flotante y errores

Métodos Numéricos 2026 · IMERL

| Los ejercicios 10 y 12 son los entregables de este práctico.

Punto flotante

Ejercicio 1 (Constantes de calculadora)

Encontrar experimentalmente los siguientes valores de la calculadora:

1. El valor ε_M .
2. El mayor número representable.
3. El menor número positivo representable.

```
% Constantes de la aritmética de punto flotante
```

Ejercicio 2 (Precisión simple)

El formato de precisión simple tiene 32 bits. Sus números están definidos por

$$x = \pm(1 + f)2^e,$$

donde el signo ocupa 1 bit, la mantisa ocupa 23 bits ($0 \leq f < 1$ y $2^{23}f$ es natural) y el exponente ocupa 8 bits ($-126 \leq e \leq 127$).

1. ¿Cuál es ε_M para este formato? ¿Cuáles son los números mayor y menor, en valor absoluto, que pueden representarse?
2. Sea

$$g(x) = \frac{x^3}{x - \sin(x)}, \quad x = 5 \cdot 10^{-4}.$$

Computar $g(x)$ en Octave usando precisión doble. Luego definir $y = \text{single}(x)$, computar $g(y)$ y explicar el resultado.

```
g = @(x) x.^3 ./ (x - sin(x));
```

```
x = 5e-4;
y = single(x);

% Comparar g(x) y g(y)
```

Ejercicio 3 (Problema 1.35 de Moler)

¿Qué realiza cada programa? ¿Cuántas líneas de salida devuelve? ¿Cuáles son los últimos dos valores de x que muestra?

```
x = 1; while 1+x > 1, x = x/2, pause(.02), end
x = 1; while x+x > x, x = 2*x, pause(.02), end
x = 1; while x+x > x, x = x/2, pause(.02), end
```

```
% Ejecutar cada ciclo por separado y analizar sus últimos valores
```

Ejercicio 4 (Examen de febrero de 2024)

Sea $f : \mathbb{R} \setminus \{0\} \rightarrow \mathbb{R}$,

$$f(x) = \frac{1 - \cos(x)}{x^2}.$$

1. Verificar que $\lim_{x \rightarrow 0} f(x) = \frac{1}{2}$. En adelante tomar $f(0) = \frac{1}{2}$ y asumir $f'(0) = 0$.
2. Calcular el número de condición de f en $x = 0$ y estimar el error inevitable al evaluar $f(x)$ para $x \approx 0$ con precisión doble y redondeo.
3. Explicar por qué el código siguiente devuelve resultados muy malos para 10^{-8} y 10^{-9} .
4. Reescribir $f(x)$ para evaluarla de forma numéricamente estable cerca de cero.

```
f = @(x) (1-cos(x))./(x.^2);
valores = [f(1e-5), f(1e-6), f(1e-7), f(1e-8), f(1e-9)]

% Definir aquí una versión numéricamente estable
```

Errores

Ejercicio 5 (Raíces de polinomio)

Hallar las dos raíces del polinomio

$$x^2 + 5x + 10^{-12}$$

con cuatro dígitos de precisión.

% Comparar la fórmula cuadrática directa con una forma estable

Ejercicio 6 (Aproximación de la derivada)

Para $f: I \rightarrow \mathbb{R}$ de clase C^∞ , aproximar $f'(a)$ mediante el cociente incremental centrado

$$\delta_a(h) = \frac{f(a+h) - f(a-h)}{2h}.$$

1. Usar Taylor para estimar $|\delta_a(h) - f'(a)|$.
2. Para $f(x) = \sin(5x)$ y $a = 1$, usar $h = 0.1^k$, $k = 0, 1, \dots, 20$. Graficar con loglog el error absoluto en función de h y explicar el comportamiento.
3. Estimar el h óptimo considerando errores de truncamiento y redondeo, y compararlo con el experimento.

```
f = @(x) sin(5*x);
a = 1;
h = 0.1 .^ (0:20);
aprox = (f(a+h) - f(a-h)) ./ (2*h);
exacta = 5*cos(5*a);
error = abs(aprox - exacta);

loglog(h, error, 'o-');
grid on;
xlabel('h'); ylabel('error absoluto');
```

Ejercicio 7 (Algoritmo babilonio)

Para aproximar $\sqrt{2}$, considerar $a_0 = 2$ y

$$a_{n+1} = \frac{a_n}{2} + \frac{1}{a_n}.$$

1. Demostrar que $a_n > 0$ para todo n .
2. Probar que a_n está acotada inferiormente por $\sqrt{2}$, es decreciente y converge a $\sqrt{2}$.
3. Para $e_n = |a_n - \sqrt{2}|$, hallar $0 < C < 1$ tal que $e_{n+1} \leq C e_n^2$.
4. Demostrar que $a_n \in \mathbb{Q}$ y, por tanto, $a_n \neq \sqrt{2}$.

5. Implementar la recurrencia en la forma

$$a_{n+1} = \left(\frac{a_n^2 + 2}{2} \right) \frac{1}{a_n},$$

analizar e_n y hallar el primer $n_0 > 0$ para el cual la máquina confunde a_{n_0} con $\sqrt{2}$.

```
a = 2;
n = 0;
while true
    siguiente = ((a^2 + 2)/2) / a;
    n = n + 1;
    if siguiente == sqrt(2)
        break;
    endif
    a = siguiente;
endwhile

n0 = n
```

Ejercicio 8 (Cancelación catastrófica y desborde)

1. Reescribir

$$\lim_{n \rightarrow +\infty} \int_n^{n+1} \log(x) dx$$

para evitar cancelación catastrófica.

2. Reescribir $e^x / (e^x + 1)$ para evaluarla con valores grandes de x sin desborde.
3. Analizar los problemas del cociente $[\cos(x + h) - \cos(x)]/h$ y reescribirlo para aproximar la derivada de forma más estable.

Ejercicio 9 (Una recurrencia inestable)

Los números

$$p_n = \int_0^1 x^n e^x dx$$

satisfacen $p_1 > p_2 > \dots > 0$ y

$$p_{n+1} = e - (n+1)p_n, \quad p_1 = 1.$$

1. Demostrar la recurrencia.
2. Generar los primeros 20 valores y explicar por qué no respetan la desigualdad.
3. Tomar $p_{20} = 1/8$ y usar la recurrencia hacia atrás para calcular $p_{19}, \dots, p_1$. Comparar estabilidad. Repetir con $p_{20} = 20$ y $p_{20} = 100$.

```
% Recurrencia hacia adelante
p = zeros(1, 20);
p(1) = 1;
for n = 1:19
    p(n+1) = exp(1) - (n+1)*p(n);
endfor

p
```

Ejercicio 10 (Otra recurrencia inestable) · Entregable

Para $n \in \mathbb{N}$, sea

$$I_n = \int_0^1 \frac{x^n}{5+x} dx.$$

1. Verificar que $I_0 = \log(6/5)$, $I_n \geq 0$ y

$$I_n + 5I_{n-1} = \frac{1}{n}.$$

2. Implementar la recurrencia hacia adelante para calcular I_{30} . Analizar si el resultado es confiable mediante error hacia adelante.
3. Comenzar con $I_{100} \approx 0$ y usar la recurrencia hacia atrás para calcular I_{30} . Repetir con $I_{100} = 0.1$ e $I_{100} = 1$ y explicar los resultados.

```
% Recurrencia hacia adelante
I = zeros(1, 31);
I(1) = log(6/5);
for n = 1:30
    I(n+1) = 1/n - 5*I(n);
endfor
I30_adelante = I(31)

% Implementar aquí la recurrencia hacia atrás desde I_100
```

Ejercicio 11 (Área de un triángulo)

La fórmula de Herón es

$$A(T) = \sqrt{p(p-a)(p-b)(p-c)}, \quad p = \frac{a+b+c}{2}.$$

1. Escribir $A = \text{area}(a,b,c)$, validando positividad y desigualdad triangular.
2. Para el triángulo rectángulo $a = 1$, $b = \delta$, $c = \sqrt{1 + \delta^2}$, usar $\delta = 10^{-n}$, $n = 1, \dots, 16$, y calcular errores relativos.
3. Repetir para el triángulo isósceles $a = 2$, $b = c = \sqrt{1 + \delta^2}$ y explicar la diferencia.

```
function A = area_triángulo(a, b, c)
    if a <= 0 || b <= 0 || c <= 0 || a+b <= c || a+c <= b || b+c <= a
        error('Los lados no forman un triángulo válido');
    endif
    p = (a+b+c)/2;
    A = sqrt(p*(p-a)*(p-b)*(p-c));
endfunction
```

Ejercicio 12 (Extrapolación de Richardson) · Entregable

Supóngase

$$p^* = p_0(h) + a_0 h^{k_0} + a_1 h^{k_1} + \dots, \quad 0 < k_0 < k_1 < \dots.$$

Se combina $p_0(h)$ con $p_0(h/t)$ para obtener $p_1(h) = \alpha p_0(h) + \beta p_0(h/t)$.

1. Hallar α y β y verificar

$$p_1(h) = \frac{t^{k_0} p_0(h/t) - p_0(h)}{t^{k_0} - 1}.$$

2. Dado que $(1+h)^{1/h}$ aproxima e con orden uno, usar Richardson con $t = 2$ para obtener una aproximación de orden dos.
3. Para $h = 10^{-2}, 10^{-3}, 10^{-4}$, comparar en Octave los errores de ambos métodos.

```
h = [1e-2, 1e-3, 1e-4];
p0 = @(z) (1+z).^(1./z);
p1 = 2*p0(h/2) - p0(h);

error_p0 = abs(p0(h) - exp(1))
error_p1 = abs(p1 - exp(1))
```

Ejercicios opcionales

Ejercicio 13 (Derivada con complejos)

Sea $f : I \rightarrow \mathbb{R}$ de clase C^∞ . Para $h > 0$:

1. Aproximar $f(a + ih)$ mediante Taylor.
2. Estimar el error de

$$\Delta_a(h) = \text{Im} \left(\frac{f(a + ih)}{h} \right)$$

como aproximación de $f'(a)$.

3. Repetir las partes experimentales del Ejercicio 6 para $\Delta_a(h)$ y explicar por qué es más robusta computacionalmente.

```
f = @(x) sin(5*x);  
a = 1;  
h = 0.1 .^ (0:20);  
aprox = imag(f(a + 1i*h)) ./ h;  
error = abs(aprox - 5*cos(5*a));  
loglog(h, error, 'o-'); grid on;
```

Ejercicio 14 (Serie exponencial)

Calcular la exponencial mediante

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

1. Usar el programa dado para sumar hasta $n = 100$ con $x = -20, -19, \dots, 20$.
2. Graficar con semilogy el error relativo respecto de exp, localizar los peores resultados y justificarlos.
3. Proponer una forma más precisa para los valores problemáticos.

```
x = -20:20;  
suma = ones(size(x));  
t = x; n = 1;  
while n < 100  
    suma = suma + t;  
    n = n + 1;  
    t = t .* x / n;  
endwhile  
  
error_relativo = abs((suma - exp(x)) ./ exp(x));
```

```
semilogy(x, error_relativo, 'o-'); grid on;
```

Ejercicio 15 (Overflow en sumas)

1. Analizar cuándo la suma secuencial $\sum_{k=1}^n x_k$ puede sufrir cancelación catastrófica o desborde. Construir un ejemplo de cada caso.
2. Analizar el escalado siguiente y explicar cuándo evita un desborde:

```
[~, ~, m] = find(max(abs(x)));  
x = x/m;  
suma = 0;  
for k = 1:n  
    suma = suma + x(k);  
end  
suma = m*suma;
```

3. Usar la misma idea para calcular establemente la norma euclídea de vectores $x \in \mathbb{R}^n$ con n grande.

```
% Implementar una norma euclídea estable mediante escalado
```